《状态机应用》

《状态机应用》 1
一：状态机 3
1.1 状态机定义 3
1.2 状态机分类 3
1.3 如何设计状态机 4
1.3.1 交通灯控制系统设计案例 5
1.3.2 状态机设计流程 6
1.3.3 序列检测器设计案例 9
二：状态机结构 11
2.1 结构统一（一段式） 11
2.2 结构二（二段式） 12
2.3 结构三（三段式） 13
2.4 状态电路结构选择 15
2.5 自动售货机设计案例 16
三：状态机有关应用 18
3.1 按键消抖应用 18
3.1 机械按键 18
3.1.1 按键抖动 19
3.1.2 按键消抖设计案例 20
3.2 PWM 应用 22
3.2.1PWM 原理 22
3.2.2LED 灯亮度控制设计案例 23

一：状态机

在数字电路设计中，状态机是非常重要的一种电路设计，熟练掌握状态机是一个数字电路设计者必须具备的能力，对于 HDL 设计，写好状态机是一种能力。

1.1 状态机定义

定义 1:是电**路运行状态控制**的电路结构，任何数字电路通电后都是按照我们预设的逻辑关系**自动运行**，如果我们需要让电路按照我们的要求运行，这就涉及到运行状态如何管理的问题，在电路设计只管理运行状态依赖于一个特殊的电路结构，即状态机。

定义 2：从电路系统的角度看，它属于有限自动机（FA）的一种，叫做有限状态机，系统表示如下：

状态变量

R = FSM(S,state)
state = sm(s,t)

s：状态机电路输入信号
r: 状态机电路输出信号
state:状态变量
t:时间

从定义 2 可以看出一个状态机电路结构包括 3 要素：**输入信号**，**输出信号**，**状态变量**。

1.2 状态机分类

状态机的分类参考常规系统定义：

R = FSM(S,state)
state = sm(s,t)

在实际电路中该定义分 2 种情况：

1：R = FSM(S,state)中 s 在状态机中不需要，我们把这种状态机叫走 moore 状态机，可表示为:

R = FSM(state)

state = sm(s,t)

2:R = FSM(S,state)中 s 在状态机需要，我们把这种状态机叫走 mealy 状态机，可表示为:

R = FSM(s,state)

state = sm(s,t)

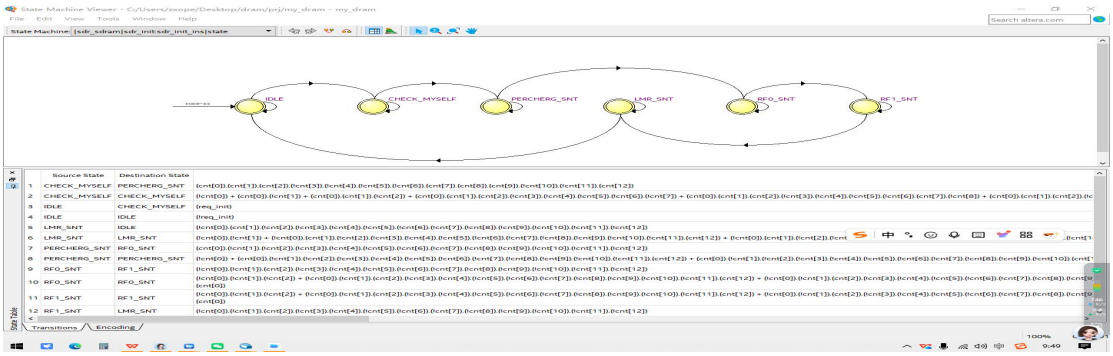
1.3 如何设计状态机

设计状态机按照其定义进行设计，从定义上来看状态机包括 2 个子电路：

1：状态变量随输入信号变化而变化的子电路系统 **state = sm(s,t)**

2：输出信号（响应信号）随状态变量，输入信号（可不需要）变化而变化的电路系统 **R = FSM(s,state)**。

状态机设计，通过图形化的方式表示，实际用通过图表表示，有 2 种方式：



1：状态图

图形符号	符号名称	符号定义
状态名字	状态变量	状态变量的符号表示，状态名字表示当状态编码
条件 →	状态转移	表示状态随输入信号条件变化而变化的一种关系
▭	状态输出	表示输出信号随输入信号（可省略）/状态变量变化而变化的一种关系

2：状态表

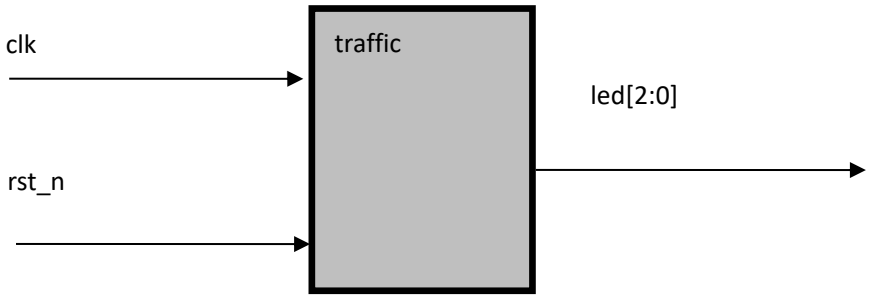
通过表格的方式表示状态机电路系统的一种方法：

当前状态	下一个状态	状态转移条件	状态输出
状态编码 0	下一个状态编码	…	…
…	…	..	…

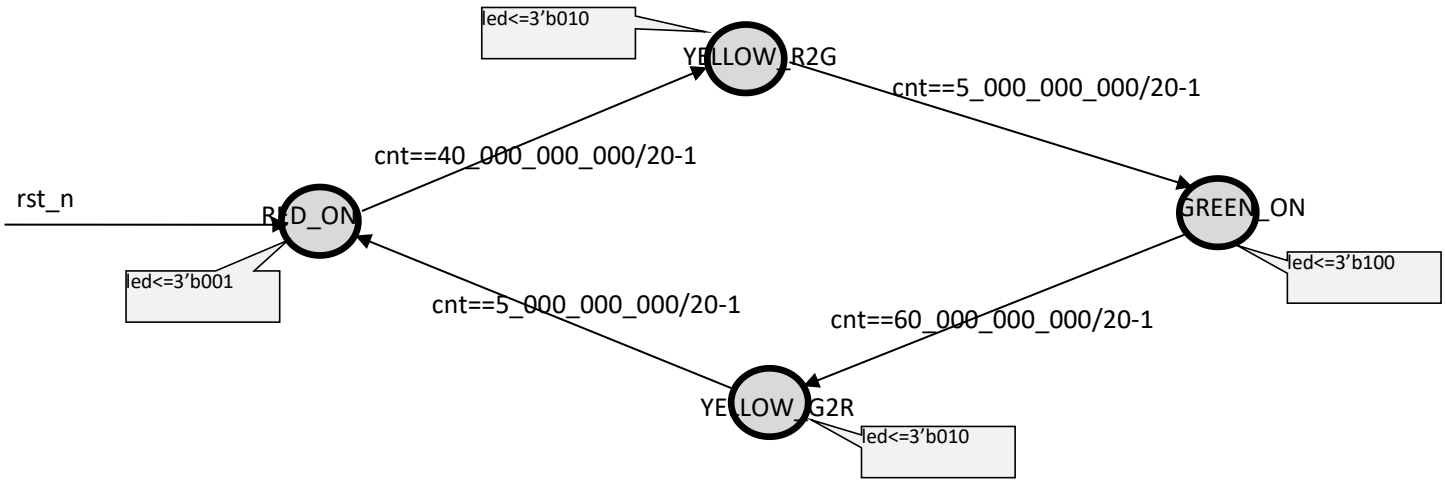
Note:在数字电路设计中一般选择状态图进行状态机的设计。

1.3.1 交通灯控制系统设计案例

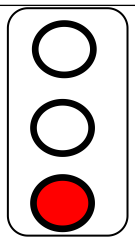
标题：交通灯控制系统设计
要求：
1：包括 3 种灯,红灯，黄灯，绿灯
2：绿灯持续时间 60s
3: 绿灯持续时间 5s
4：绿灯持续时间 40s



状态图：
state = sm(s,t)
R = FSM(s,state)



端口说明

端口信号名	描述
系统相关	
clk	系统时钟，默认 mhz
rst_n	系统复位，低电平有效
交通灯	
led[2:0]	交通控制灯 led[2]：绿灯 led[1]：黄灯 led[0]：红灯 

1.3.2 状态机设计流程

数字电路设计中经常用到状态机，如何正确设计状态是我们关心的问题，以下是状态机的标准设计流程;

第 1 步：确定状态变量

根据我们的设计要求确定我们使用的状态，比如交通灯控制用到 4 个状态。

RED_ON：该状态只允许红灯亮，其余灯灭

GREEN_ON:该状态只允许绿灯亮，其余灯灭、

YELLOW_R2G:该状态只允许黄灯亮，其余灯灭，且为红灯变化为绿灯

YELLOW_G2R:该状态只允许黄灯亮，其余灯灭，且为绿灯变化为红灯

第 2 步：状态变量编码

在电路设计中状态变量用信号表示，不同状态对应不同的信号值，即状态编码，在状态机设计中编码方式不是唯一的，公认的有 3 种编码：

1：自然二进制数码

状态变量所表示的信号的值用自然二进制数表示

如：

```
localparam   RED_ON       =2'd0,//该状态只允许红灯亮，其余灯灭
             GREEN_ON     ='d1,//该状态只允许绿灯亮，其余灯灭、
             YELLOW_R2G   =2d'2,//该状态只允许黄灯亮，其余灯灭，且为红灯变化为绿灯
             YELLOW_G2R   =2'd3;//该状态只允许黄灯亮，其余灯灭，且为绿灯变化为红灯
```

2:格雷码

格雷码是二进制编码的一种，如果状态变量变化是递增或者递减的时候能保证每次只有 1 位变化，电路更稳定。

```
localparam   RED_ON       =2'b00,//该状态只允许红灯亮，其余灯灭
             GREEN_ON     =2'b01,//该状态只允许绿灯亮，其余灯灭、
             YELLOW_R2G   =2'b11,//该状态只允许黄灯亮，其余灯灭，且为红灯变化为绿灯
             YELLOW_G2R   =2'b10;//该状态只允许黄灯亮，其余灯灭，且为绿灯变化为红灯
```

相邻 2 个数之间 1 位变化：2'b00->2'b01->2'b11->2'b10,以下是常见的格雷码二进制数对照表。

十进制数	自然二进制码	3 位格雷码	相邻编码变化位
0	000	000	-
1	001	001	最低位（第 0 位）
2	010	011	中间位（第 1 位）
3	011	010	最低位（第 0 位）
4	100	110	最高位（第 2 位）
5	101	111	最低位（第 0 位）
6	110	101	中间位（第 1 位）
7	111	100	最[illegible]（第 0 位）

3：独热码（one-hot）

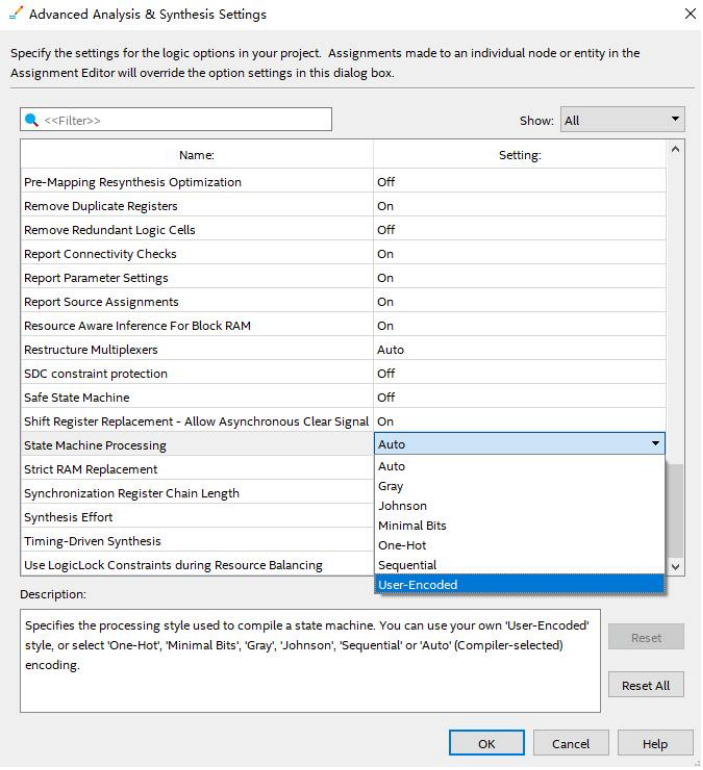
独热码本质是一个状态用一个值表示，是状态机最稳定的编码

```
localparam   RED_ON       =4'b0001,//该状态只允许红灯亮，其余灯灭
             GREEN_ON     =4'b0010,//该状态只允许绿灯亮，其余灯灭、
             YELLOW_R2G   =4'b0100,//该状态只允许黄灯亮，其余灯灭，且为红灯变化为绿灯
             YELLOW_G2R   =4'b1000;//该状态只允许黄灯亮，其余灯灭，且为绿灯变化为红灯
```

针对这三种编码，以下是三者主流编码的区别：

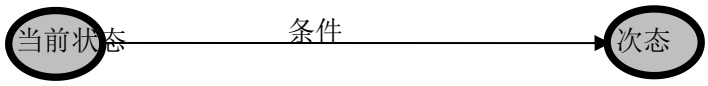
编码方式	状态数 N - 编码位数 n	寄存器资源	组合逻辑（译码）	时序性能
自然二进制码	$n = \lceil \log_2 N \rceil$	最少	中等	一般
独热码	$n = N$	最多	最少（极简单）	最优（最快）
格雷码	$n = \lceil \log_2 N \rceil$	中等（和二进制一致）	中等偏复杂	较好

在综合过程中，我们也可以指定综合所采用的优先编码方式，比如 QUARTUSS II



第 3 步：状态转移

根据实际情况表示状态变量不同状态变化的一种关系



当前状态：在设计状态机的时候，比如状态图箭头的尾端为当前状态

次态：在设计状态机的时候　，比如状态图箭头指向的状态为次态，又叫做下一状态

第 4 步：状态化简

在状态机的设计中，原则上状态越少也好，有的时候我们所设计的状态机可以优化状态变量，合并某些撞他，这样使得状态机的运行效率更高。

状态机化简没有统一的方式方法，需要我们自己去判断。

第 5 步：状态输出

根据自己需要确定所输出的信号，同时确定是 moore 还是 mealy。

1.3.3 序列检测器设计案例

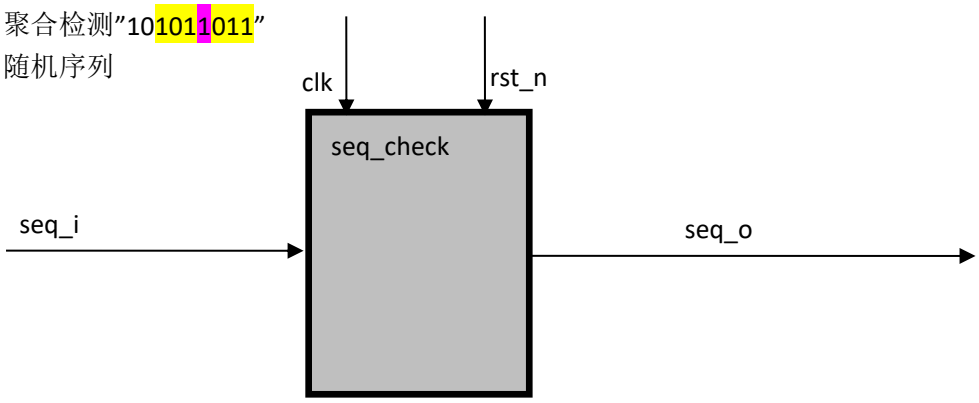
标题：序列检测器设计

要求：

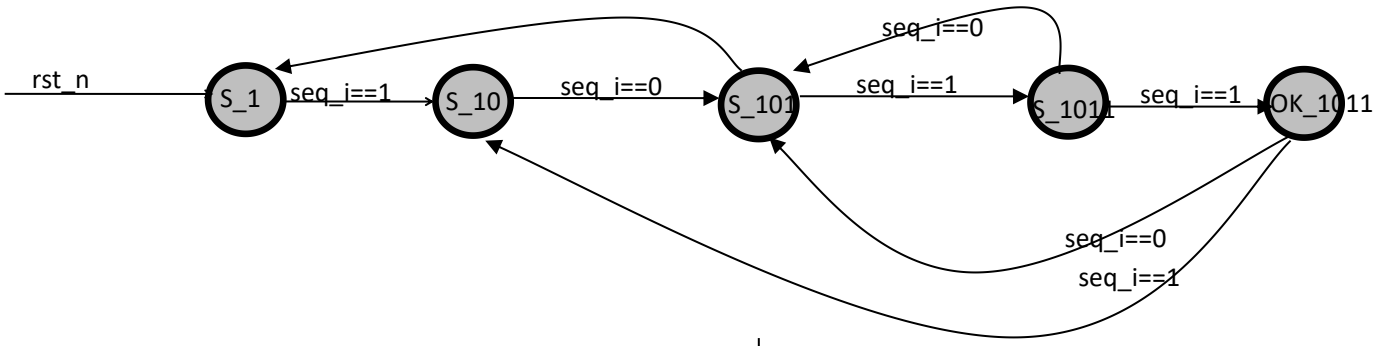
1：被检序列为“1011”

2:　聚合检测”101011011”

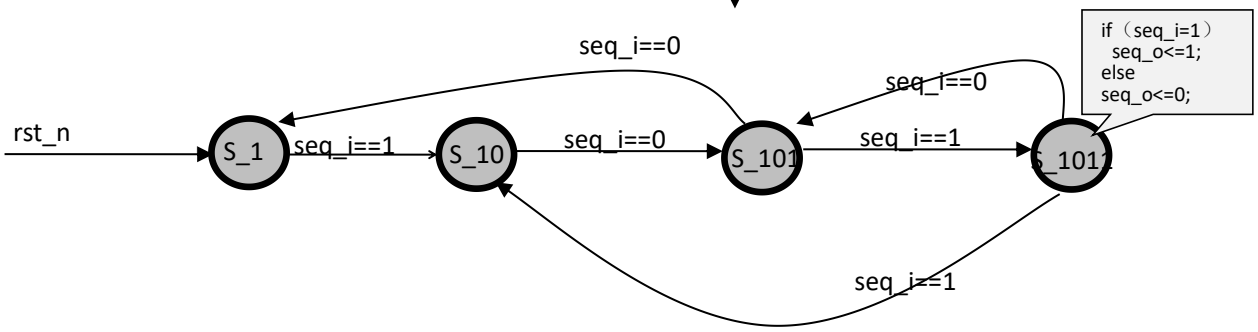
3:　随机序列



状态机设计：

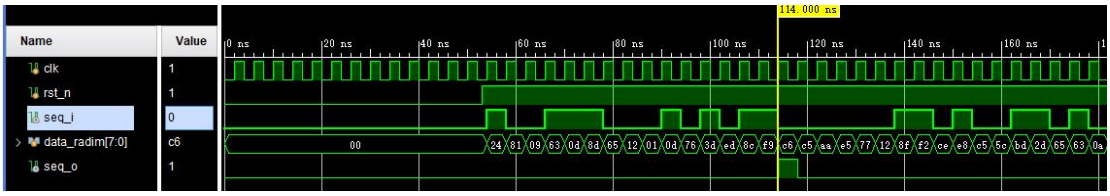


化简



端口说明

端口信号名	描述
系统相关	
clk	系统时钟，默认 50mhz
rst_n	系统复位，低电平有效
随机序列输入	
seq_i	随机待检测的序列
输出检测标志	
seq_o	当检测到“1011”的时候程序一个 clk 周期的高电平，其余为低电平 Note:本序列检测为聚合检测



二：状态机结构

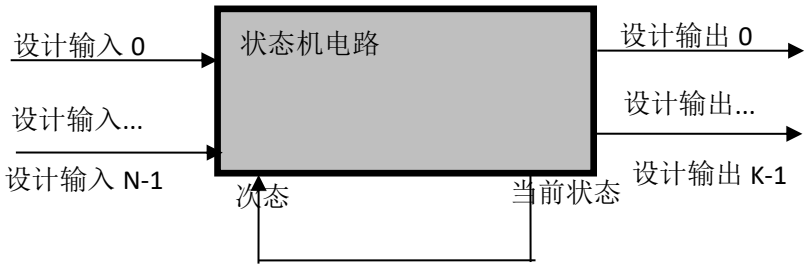
状态机是数字电路设计中一种重要的电路，写好状态机是必备的技能，状态机设计种种重是稳定，稳定是状态机设计的前提，设计中为了更稳定，除了状态编码之外，电路结构也是影响状态机稳定的重要因素。

每个人设计状态机不一样，综合器综合出来的电路也不一样，为了统一状态机电路设计，，我们需要明确综合器我们选择哪种电路结构。

目前公认的状态机结构有 3 种，其它的任何电路结构属于自定义结构，下来我们就这 3 种结构进行详细讨论。

2.1 结构统一（一段式）

一段式是状态机的电路结构之一，为标准结构，也是默认的状态机电路结构，符合常规电路设计思维，容易理解。



从电路结构上来看，整个状态是一个独立的整体，包括设计输入，设计输出，状态变量。
判断一段式依据：整个状态机电路是一个整体，与多少个控制语句 always 无关

一段式	备注
判断依据	1：状态机是一个整体 2：常规设计电路
HDL 规则	当前状态和次态使用同一信号

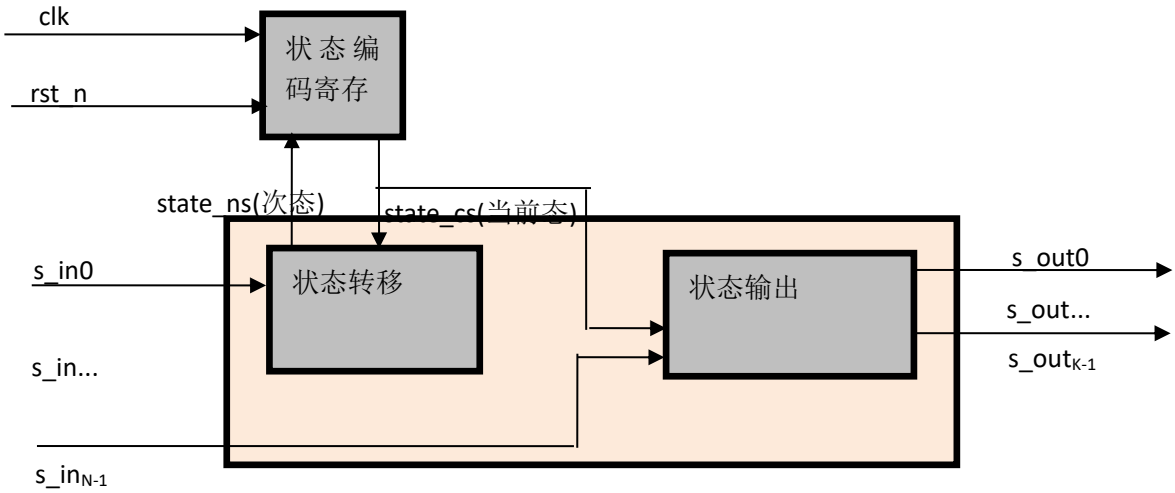
2.2 结构二（二段式）

二段式是状态机的一种电路结构，属于非常规电路设计，是一段式结构的基础上对电路结构进行了优化衍生出来，理论上优于一段式，但是实际也并非完全如此。

二段式将整个状态机工结构实现上分为 2 个独立的个体，我们从电路实现的角度去拆分电路，我们不难发现状态机大体由 3 部分核心电路构成：状态编码寄存，状态转移，状态输出，二段式也是从这样的出发点对电路结构进行分割，最终分为 2 部分：

结构部分 1：状态编码寄存，保存当前编码值，要求为时序电路。

结构部分 2: 状态转移（输出次态值）和状态输出为同一个整体电路，要求组合电路。



由于二段式是一种固定的电路结构，HDL 编码也是固定的格式，比如 verilog:

```
module      xxxx
(
            input       clk,
            input       rst_n
            .....
);
reg [....]sate_cs,state_ns;//定义状态编码
//第一段编码格式/////////////////////////////////
always      @(posedge       clk       or       negedge       rst_n)
            if(rst_n==1'b0)
                  sate_cs      <=S0;
            else
                  sate_cs      <=state_ns;

//第 2 段编码格式
```

```
always      @(*)begin
            state_ns     =     S0;
            case(sate_cs)
                    S0:...;
                    S1...;
                    ...
             endcase
end

always      @(*)begin

            case(sate_cs)
                    S0:dout=...;
                    S1:dout=....;
                    ...
             endcase
end



endmodule
```

二段式判断依据：是否符合二段式电路要求，最直接判断是否符合其二段式编码规范

2.3 结构三（三段式）

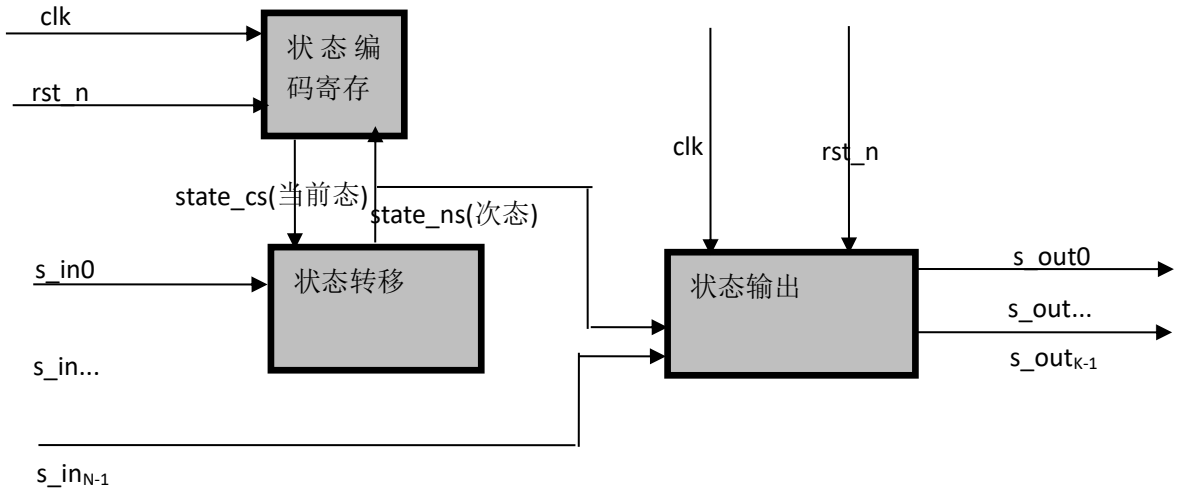
三段式是状态机的一种电路结构，属于非常规电路设计，是二段式结构的基础上对电路结构进行了优化衍生出来，理论上优于二段式，但是实际也并非完全如此。

三段式将整个状态机结结构实现上分为 3 个独立的个体，我们从电路实现的角度去拆分电路，我们不难发现状态机大体由 3 部分核心电路构成：状态编码寄存，状态转移，状态输出，三段式也是从这样的出发点对电路结构进行分割，最终分为 3 部分：

结构部分 1：状态编码寄存，保存当前编码值，要求为时序电路。

结构部分 2: 状态转移（输出次态值），要求组合电路。

结构部分 3：状态输出为一个电路部分，要求次态输出，为时序电路



由于三段式是一种固定的电路结构，HDL 编码也是固定的格式，比如 verilog:

```
module      xxxx
(
            input       clk,
            input       rst_n
            .....
);
reg [....]sate_cs,state_ns;//定义状态编码
//第一段编码格式/////////////////////////////
always      @(posedge       clk       or       negedge       rst_n)
            if(rst_n==1'b0)
                    sate_cs      <=S0;
            else
                    sate_cs      <=state_ns;

//第 2 段编码格式
always      @(*)begin
            state_ns     =     S0;
            case(sate_cs)
                    S0:...;
                    S1...;
                    ...
            endcase
end
//第 3 段编码格式

always      @(posedge       clk       or       negedge       rst_n)
            if(rst_n==1'b0)
                    dout<=...;
```

```
        case(sate_ns)
            S0:dout<=...;
            S1:dout<=....;
            ...
        endcase



endmodule
```

三段式判断依据：是否符合三段式电路要求，最直接判断是否符合其三段式编码规范

2.4 状态电路结构选择

对于状态的设计，当我们 hDL 编码的时候，二段式，三段式都有固定的格式，并且用 2 个信号来表示状态变量，按照固定的格式编写程序即可，如果用 1 个信号表示状态变量，即一段式。

✧ HDL 关键判断依据

状态机电路结构	核心特点
一段式	状态变量用 1 个信号表示
二段式	1：状态变量用 2 个信号表示，分别为当前状态和次态 2：编码格式固定
三段式	1：状态变量用 2 个信号表示，分别为当前状态和次态 2：编码格式固定

✧ 特点解析

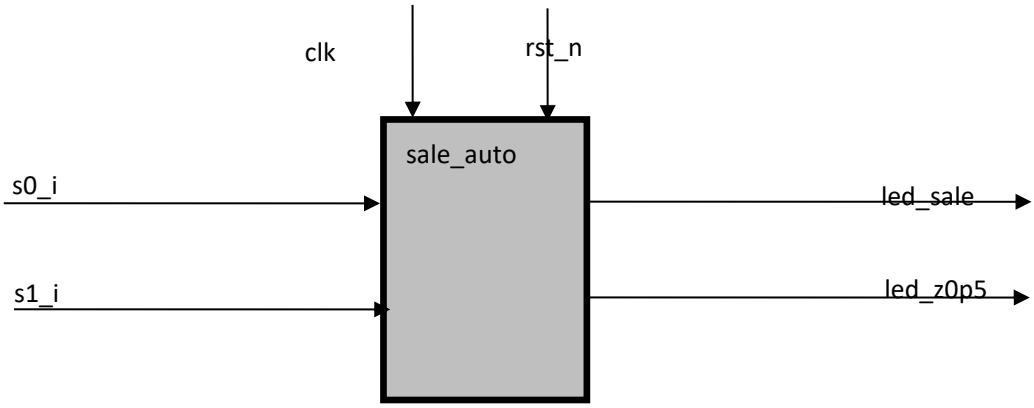
状态机电路结构	特点
稳定性	三段式>二段式>一段式
易理解	一段式>二段式>三段式
资源	三段式>二段式>一段式
简洁程度	一段式>二段式>三段式

✧ 选择原则

选择原则	最终目标
理论	三段式为工业标准
实际	状态少：优先 1 段式 状态多：优先 3 段式

2.5 自动售货机设计案例

标题：自动售货机
要求：
1：售货机支持 5 元和 1 元硬币传感器信号输入
2：所有的货物售价都为 1.5 元
3：支持找零，出货
4：用一段式，二段式，三段式分别实现，下班卡验证

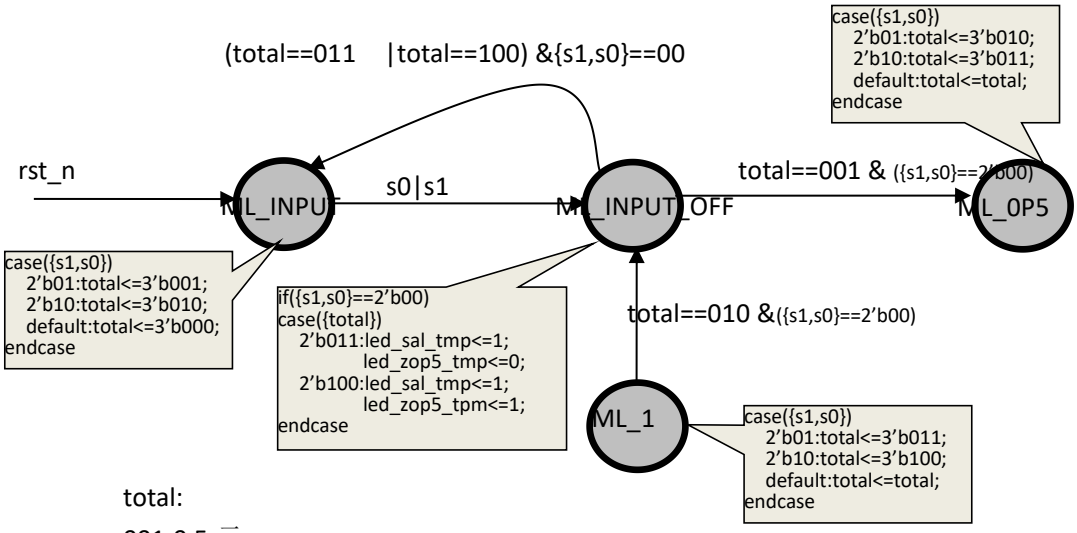


端口说明

端口信号名	描述
系统相关	
clk	系统时钟，默认 100mhz
rst_n	系统复位，默认低电平有效
传感器输入信号	
s0_i	0.5 元硬币投币 ，默认通过按键模拟
s1_i	1 元硬币投币，默认通过按键模拟
出货找零	

led_sale	当出货的时候 led 点亮 10s
led_z0p5	当找零的时候 led 点亮 10s

状态机设计



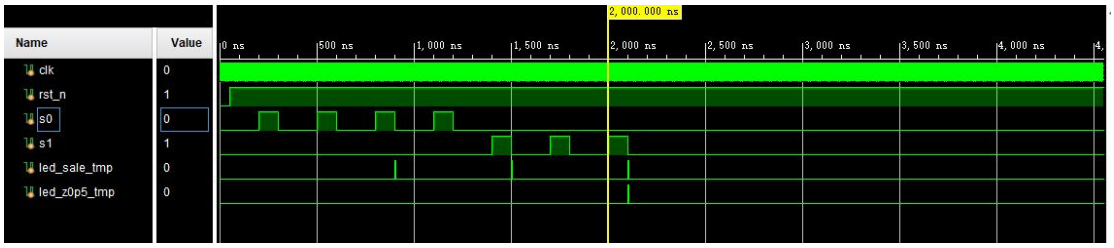
total:

001:0.5 元

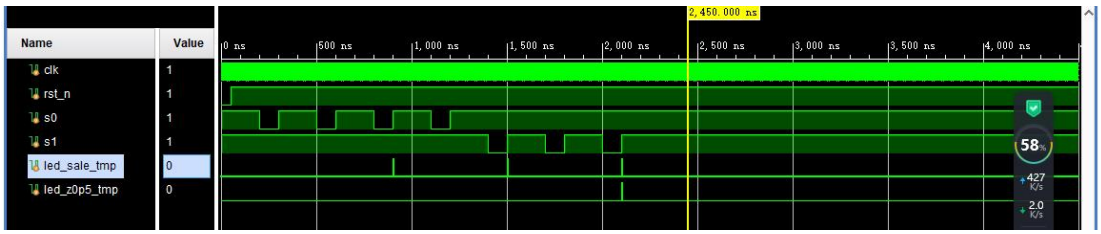
010:1 元

011:1.5 元

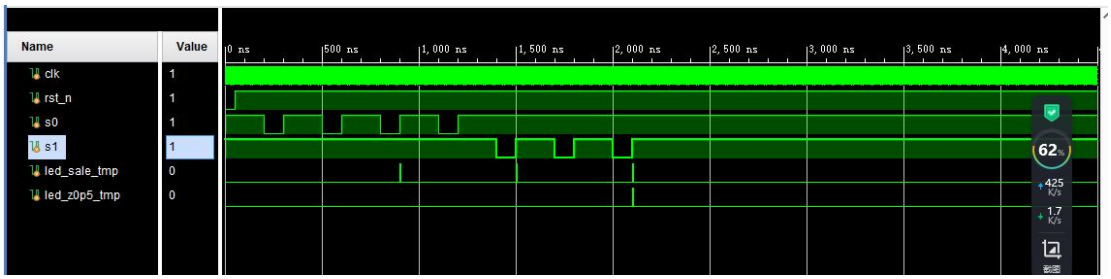
100：2 元



三段式仿真图



二段式仿真图



三：状态机有关应用

3.1 按键消抖应用

在电路设计中按键是一种简单的电路，这里我就独立按键在数字电路设计中的应用进行探索。

按键属于一种信息输入交互类的设备，按键根据物理特性归纳起来有 2 类：

1：虚拟按键

所谓虚拟按键没有物理实体的一种按键，比如软件输入法，搜狗输入法，百度输入法

2：机械按键

所谓的机械按键是由物理实体的一中按键，比如 USB 键盘上的按键，独立按键。

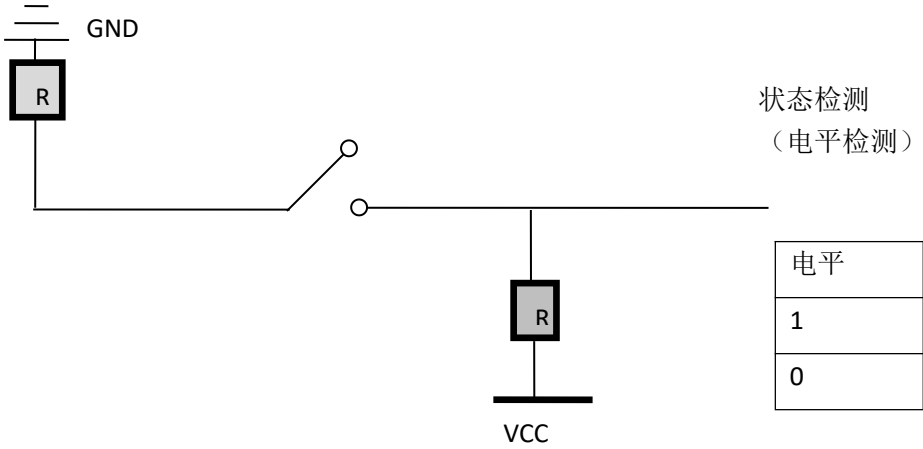
在数字电路设计中，直接与硬件交互的为机械按键，这里我们重点研究机械按键。

3.1 机械按键

按键在实际应用中是通过按键的 4 种状态被赋予特殊的意义，**通过这 4 种状态参与电路设计中**，具体 4 种状态如下;

按 键状态	说明	对应状态检测电平
稳 定按下	按键一直被按下	通常会出检测电路，为“0”
空闲	按键一直未被按下	通常会出检测电路，为“1”
按下	按键刚被按下的一段时间	通常检测电平为“下降沿”
抬起	按键刚被按下的一段时间	通常检测电平为“上升沿”

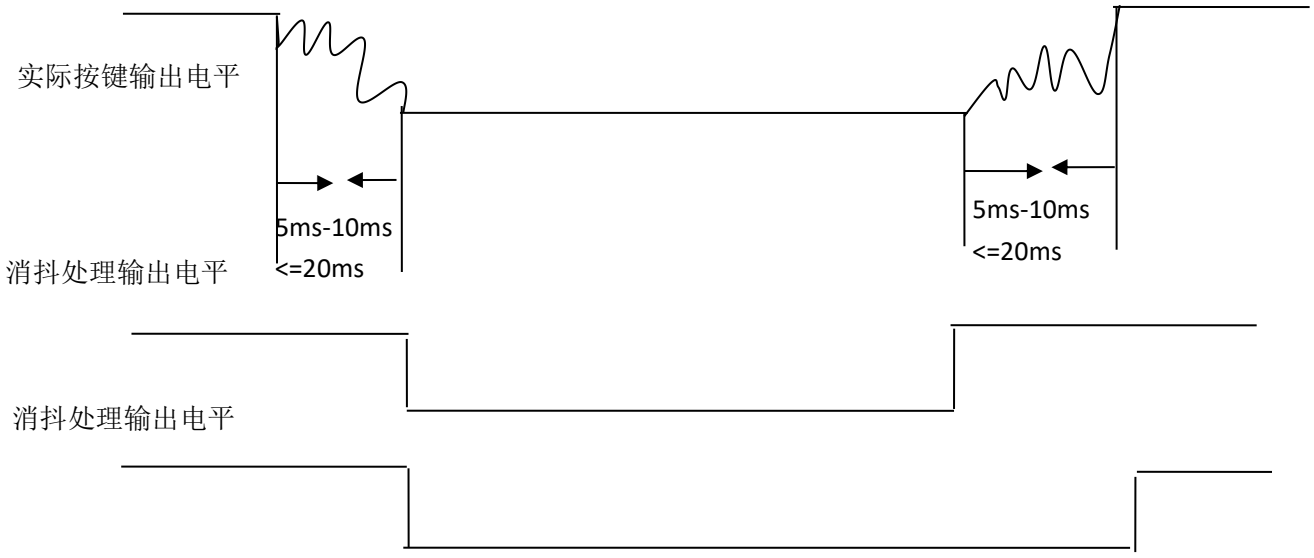
NOte:按键状态通过输出电平，检测电平达到检测状态的目的



电平	解释
1	未按下
0	按下

3.1.1 按键抖动

任何按键都是 2 个机械体通过分离或者咬合影响输出电平，在分离或者咬合的时候存在半接触的情况，这种半接触状态对输出电平有着直接的影响，造成电压不稳定，从而电平不稳定，我们把这一段时间输出的电平叫做抖动。



由于任何机械按键必然有抖动，所以要进行消抖处理，我们把经过消抖处理输出稳定电平的过程叫做按键消抖。

3.1.2 按键消抖设计案例

标题：按键消抖程序设计

要求：

1：使用 s0,s1 独立按键 2 个

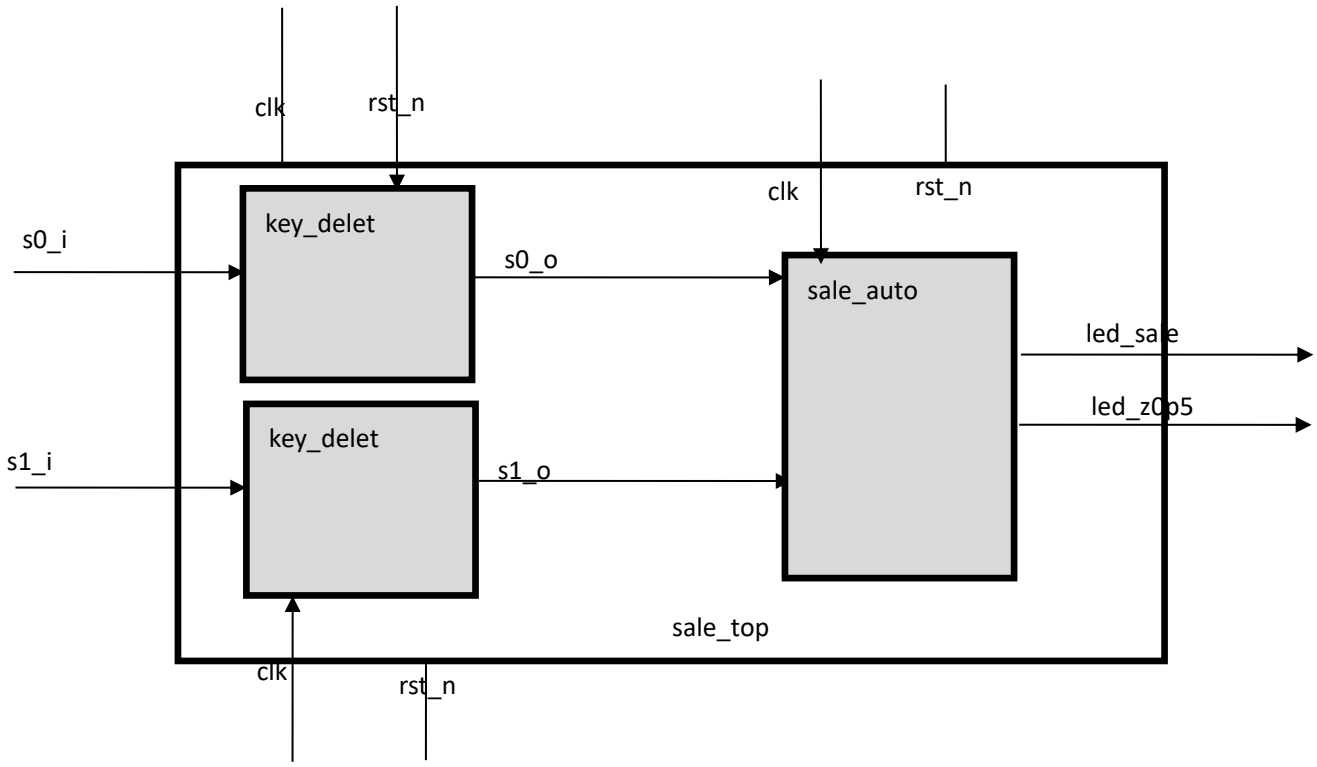
2：利用 s0，s1 作为自动收货区投币传感器信号

按键	按键状态	传感器信号信息
s0	↓	投币 0.5 元
s1	↓	投币 1 元

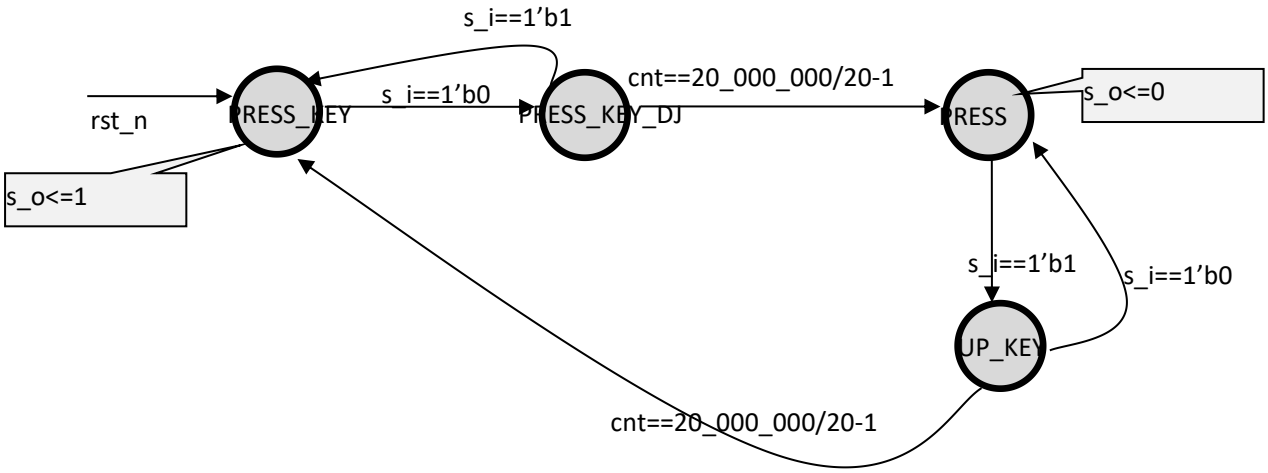
3：所有物品价格均为 1.5 元

4：支持出货，找零

5：默认一段式状态机



消抖电路程序设计状态



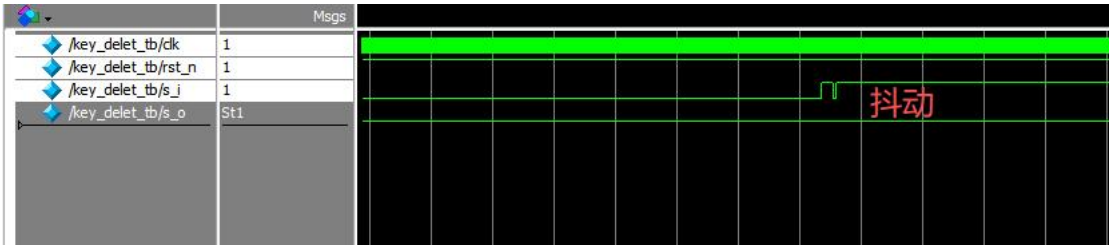
边沿检测

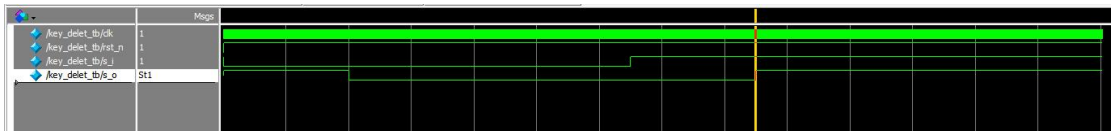
```
reg        s_o1,s_o2;
always     @(posedge     clk     )begin
           s_o1<=s_o;
           s_o2<=,s_o1;
end

wire       f_s_o;//当 f_s_o==1 表示检测到 s_o 的下降沿
assign     f_s_o=    {s_o2,s_o1}==2'b10;
```

消抖程序模块端口说明

端口信号名	描述
系统相关	

clk	系统时钟，默认 mhz
rst_n	系统复位，低电平有效
独立按键检测电平输入	
s_i	独立按键检测电平，低电平按下，有抖动
独立按键检测电平输出	
s_o	独立按键检测电平，低电平按下，无抖动

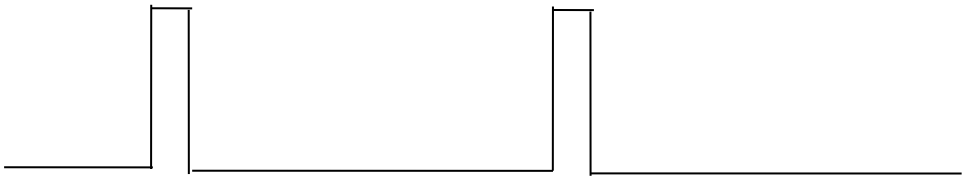




3.2 PWM 应用

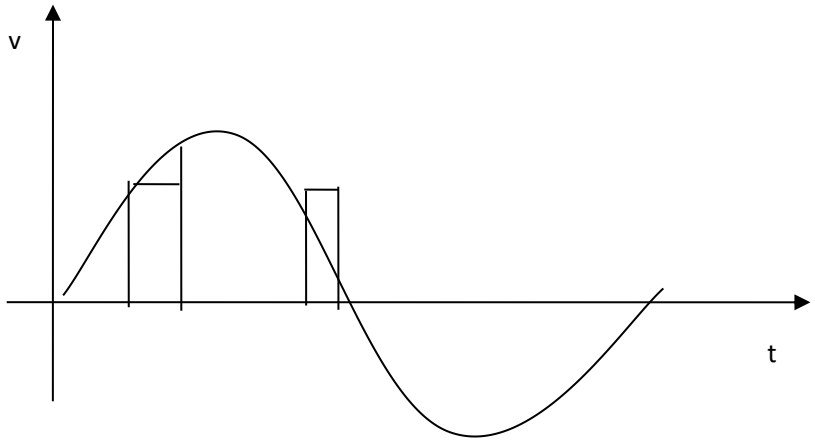
在数字设计中有的设备通过模拟信号控制，比声音大小，led 灯的亮度，对着这种应用场景在数字应用中通过 PWM 实现。

PWM 是数字领域实现模拟信号控制的有效方式，利用了面积大更小原则，通常 PWM 脉冲是周期。



脉冲信号

3.2.1PWM 原理



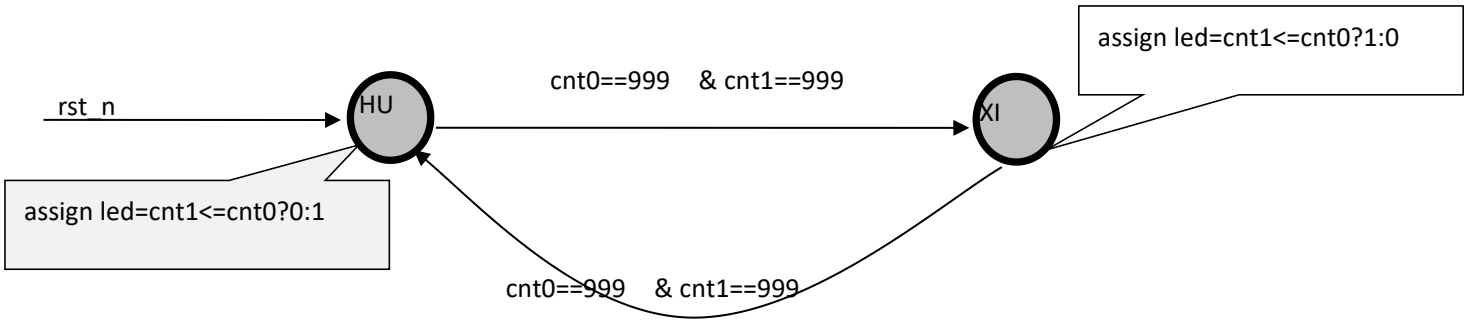
面积等效：用规则的不同宽度的脉冲信号面积等效不规则的连续信号的面积。

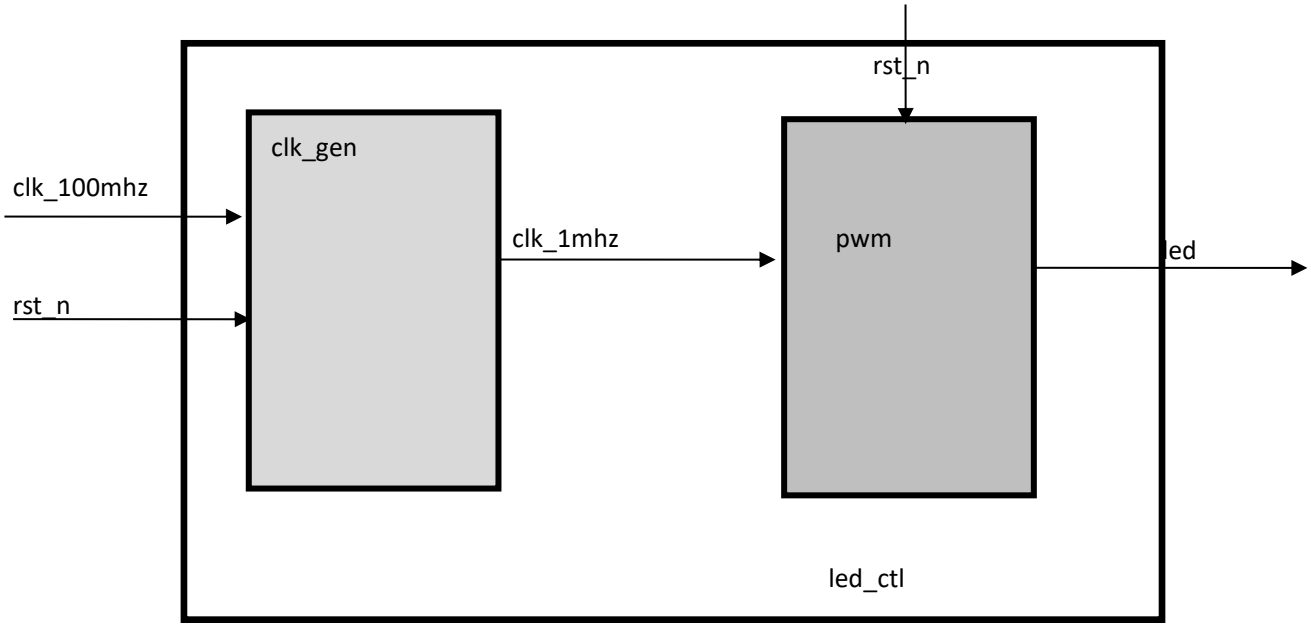
在时间轴上将信号平均分成 N 等分，每个等分用高度相同，宽度不同的脉冲信号等效连续信号，这就是所谓的 PWM。

3.2.2LED 灯亮度控制设计案例

标题：呼吸等设计
要求：
1：灯由暗到亮在到暗渐变变化
2：采用数字信号控制
3：暗->亮-》暗　2s 循环显示，即呼用 1s,吸用 1s

实现原理
呼：1s 时间内平均划分 N 等分，即一个脉冲信号的周期，每个周期的脉冲宽度逐渐变宽
吸：1s 时间内平均划分 N 等分，即一个脉冲信号的周期，每个周期的脉冲宽度逐渐变窄





if（cnt1 ==　999）cnt1<= 0;else cnt1<=cnt1+1;

if（cnt1 ==　999if(cnt0==999)cnt0<= 0;else cnt0<=cnt0+1;

asisign　　cnt　1<=cnt1　?　1’b1:1’b0;
　　cnt　1<=cnt1　?　1’b0:1’b1;